Πανεπιστήμιο Πειραιώς

Τεχνική Αναφορά: Ανάλυση Δεδομένων Accidents (MongoDB)

- **Ονοματεπώνυμο:** Θεόδωρος Τσούτσουρας
- **Αριθμός Μητρώου:** me25040

- **Τίτλος Μαθήματος:** Διαχείριση Δεδομένων για Σχεσιακές και μη Σχεσιακές Βάσεις Δεδομένων
- **Επιβλέπων Καθηγητής:** Ν. Κουτρουμάνης
- **Ημερομηνία:** Ιανουάριος 2026

Η παρούσα εργασία παρουσιάζει την επίλυση συνθέτων ερωτημάτων (queries) πάνω στο dataset Accidents.

In [1]:
```
import pymongo
from pymongo import MongoClient
from datetime import datetime
import json
from bson import json_util

client = MongoClient("mongodb://localhost:27017/")
db = client["test"]
collection = db["accidents"]
```

Ερώτημα 1

Ζητούμενο: Μετατροπή των μονάδων μέτρησης στο Μετρικό σύστημα και οριστική διαγραφή των παλιών πεδίων για την απλοποίηση της δομής των δεδομένων.

Μαθηματικοί Τύποι Μετατροπής:

1. **Θερμοκρασία (Fahrenheit $\rightarrow$ Celsius):**

$$T_{Celsius} = (T_{Fahrenheit} - 32) \times \frac{5}{9}$$

2. **Απόσταση (Miles $\rightarrow$ Kilometers):**

$$D_{km} = D_{miles} \times 1.609344$$

Τεχνική Υλοποίηση:

- Χρησιμοποιήθηκε η εντολή `update_many` για τη μαζική ενημέρωση των εγγράφων.
- Αρχικά, δημιουργούνται τα νέα πεδία `temperature(C)` και `distance(km)` υπολογιζόμενα δυναμικά από τα υπάρχοντα.

- Στη συνέχεια, εφαρμόζεται ο τελεστής `$unset` για την οριστική διαγραφή των πεδίων `temperature(F)` και `distance(mi)`, ώστε να μην υπάρχει πλεονασμός πληροφορίας.

In [2]:
```
collection.update_many(
    {},
    [{"$set": {"temperature(C)": {"$multiply": [{"$subtract": ["$temperature(F)"
)

collection.update_many(
    {},
    [{"$set": {"distance(km)": {"$multiply": ["$distance(mi)", 1.609344]}}}]
)

collection.update_many({}, {"$unset": {"temperature(F)": ""}})
collection.update_many({}, {"$unset": {"distance(mi)": ""}})

sample_doc = collection.find_one()

print(json.dumps(sample_doc, indent=4, default=json_util.default))
```


```
{
    "_id": "A-14820",
    "side": "R",
    "city": "Johnston",
    "state": "IA",
    "give_way": false,
    "stop": false,
    "location": {
        "type": "Point",
        "coordinates": [
            -93.69785,
            41.6528
        ]
    },
    "time": {
        "$date": "2017-01-01T00:44:41Z"
    },
    "temperature(C)": -5.0,
    "distance(km)": 0.38302387200000004
}
```


Ερώτημα 2

Ζητούμενο: Ο υπολογισμός του πλήθους των ατυχημάτων που πληρούν ταυτόχρονα συγκεκριμένα γεωγραφικά, χρονικά και κυκλοφοριακά κριτήρια.

Κριτήρια Φιλτραρίσματος:

- Τοποθεσία: Ακτίνα **100 χλμ** γύρω από το σημείο με συντεταγμένες `[-86.157330, 39.768535]`.
- Χρονικό Διάστημα: Ατυχήματα μεταξύ **15 Ιουνίου 2017** και **15 Οκτωβρίου 2017**.
- Σήμανση: Να υπάρχει πινακίδα **Stop** ή **Give Way**.

Τεχνική Υλοποίηση: Για το γεωχωρικό ερώτημα χρησιμοποιήθηκε ο τελεστής `$geoWithin` με τη μέθοδο `$centerSphere`. Σημειώνεται ότι η ακτίνα των 100km

διαιρέθηκε με την ακτίνα της Γης (6378.1 km) για να μετατραπεί σε ακτίνια (radians), όπως απαιτεί η MongoDB.

Προτεινόμενα Indexes: Για την επιτάχυνση της εκτέλεσης του συγκεκριμένου ερωτήματος, κρίνεται απαραίτητη η δημιουργία των εξής ευρετηρίων:

- **Geospatial Index (`2dsphere`):** Στο πεδίο `location` για την υποστήριξη και επιτάχυνση του γεωχωρικού ερωτήματος ($geoWithin).
- **Compound Index:** Ένας σύνθετος δείκτης της μορφής `{ "location": "2dsphere", "time": 1 }` θα παρείχε τη μέγιστη απόδοση, καθώς θα κάλυπτε ταυτόχρονα τον γεωγραφικό περιορισμό και το χρονικό εύρος, ελαχιστοποιώντας τα έγγραφα που σαρώνει η βάση.

In [3]:
```
start_date = datetime(2017, 6, 15)
end_date = datetime(2017, 10, 15)

pipeline_q2 = [
    {
        "$match": {
            "location": {
                "$geoWithin": {
                    "$centerSphere": [
                        [-86.157330, 39.768535],
                        100 / 6378.1
                    ]
                }
            },
            "time": {
                "$gte": start_date,
                "$lte": end_date
            },
            "$or": [
                {"stop": True},
                {"give_way": True}
            ]
        }
    },
    {
        "$count": "count"
    }
]

result_q2 = list(collection.aggregate(pipeline_q2))

print(json.dumps(result_q2, indent=4, default=json_util.default))
```

```
[
    {
        "count": 7
    }
]
```

Ερώτημα 3

Ζητούμενο: Δημιουργία ιστογράμματος που απεικονίζει τη συχνότητα των ατυχημάτων ανά ημέρα της εβδομάδας, εστιάζοντας αποκλειστικά στη χρονική ζώνη της νύχτας.

Κριτήρια Φιλτραρίσματος:

- Χρονικό Φίλτρο (Νύχτα): Ως νυχτερινές ώρες ορίζεται το διάστημα από **22:00 έως 04:00**.
- Ομαδοποίηση: Τα δεδομένα ομαδοποιούνται ανά ημέρα της εβδομάδας.

Τεχνική Υλοποίηση: Η επεξεργασία υλοποιήθηκε μέσω ενός Aggregation Pipeline δύο σταδίων, με χρήση των εξής τελεστών:

- Φιλτράρισμα (`$match`): Λόγω του ότι το χρονικό παράθυρο διατρέχει την αλλαγή της ημέρας (μεσάνυχτα), χρησιμοποιήθηκε ο τελεστής `$expr` σε συνδυασμό με τον λογικό τελεστή `$or` . Η ώρα εξάγεται δυναμικά μέσω του `$hour` από το πεδίο `time` , ελέγχοντας αν ικανοποιείται η συνθήκη: `($hour ≥ 22) OR ($hour < 4)` .
- Ομαδοποίηση (`$group`): Η ομαδοποίηση πραγματοποιήθηκε βάσει της ημέρας της εβδομάδας με χρήση του τελεστή `$dayOfWeek` (το οποίο επιστρέφει αριθμό 1-7). Για τον υπολογισμό του πλήθους των ατυχημάτων ανά ημέρα, χρησιμοποιήθηκε ο συσσωρευτής `$sum` με την τιμή 1.

In [4]:
```
pipeline_q3 = [
    {
        "$match": {
            "$expr": {
                "$or": [
                    {"$gte": [{"$hour": "$time"}, 22]}, # 22:00 - 23:59
                    {"$lt":  [{"$hour": "$time"}, 4]}   # 00:00 - 03:59
                ]
            }
        }
    },
    {
        "$group": {
            "_id": {"$dayOfWeek": "$time"},
            "count": {"$sum": 1}
        }
    }
]

result_q3 = list(collection.aggregate(pipeline_q3))

print(json.dumps(result_q3, indent=4, default=json_util.default))
```

```
[
    {
        "_id": 4,
        "count": 3498
    },
    {
        "_id": 3,
        "count": 3433
    },
    {
        "_id": 1,
        "count": 4188
    },
    {
        "_id": 2,
        "count": 3774
    },
    {
        "_id": 6,
        "count": 3429
    },
    {
        "_id": 5,
        "count": 3354
    },
    {
        "_id": 7,
        "count": 3724
    }
]
```


Ερώτημα 4: (Αρχική Διερεύνηση)

Ζητούμενο: Εντοπισμός της πόλης με τη μέγιστη αριθμητική διαφορά ατυχημάτων μεταξύ της αριστερής και της δεξιάς πλευράς του δρόμου, για τις περιπτώσεις που τα ατυχήματα στα αριστερά υπερτερούν (`Left > Right`).

Τεχνική Υλοποίηση (Initial Pipeline): Για την επίλυση του προβλήματος, κατασκευάστηκε ένα pipeline 5 σταδίων:

- Ομαδοποίηση (`$group`): Τα δεδομένα ομαδοποιήθηκαν ανά Πόλη και Πολιτεία. Ταυτόχρονα, χρησιμοποιήθηκε η τεχνική "Pivoting" με τον τελεστή `$cond` :
  - Αν `side == "L"` , προστίθεται 1 στο `countL` .
  - Αν `side == "R"` , προστίθεται 1 στο `countR` .
- Υπολογισμός Διαφοράς (`$project`): Δημιουργία του πεδίου `diff` μέσω αφαίρεσης: $Count_L - Count_R$.
- Φιλτράρισμα (`$match`): Επιλογή μόνο των εγγραφών όπου η διαφορά είναι θετική (`diff > 0`), δηλαδή τα ατυχήματα στα αριστερά είναι περισσότερα.
- Ταξινόμηση & Όριο (`$sort` , `$limit`): Φθίνουσα ταξινόμηση βάσει της διαφοράς και εμφάνιση των 2 κορυφαίων αποτελεσμάτων για την αξιολόγηση των δεδομένων.

In [5]:
```
pipeline_q4_initial = [
    {
        "$group": {
            "_id": {"city": "$city", "state": "$state"},
            "countL": {"$sum": {"$cond": [{"$eq": ["$side", "L"]}, 1, 0]}},
            "countR": {"$sum": {"$cond": [{"$eq": ["$side", "R"]}, 1, 0]}}
        }
    },
    {
        "$project": {
            "diff": {"$subtract": ["$countL", "$countR"]},
            "countL": 1,
            "countR": 1,
            "city": "$_id.city"
        }
    },
    {
        "$match": {"diff": {"$gt": 0}}
    },
    {
        "$sort": {"diff": -1}
    },
    {
        "$limit": 2
    }
]

result_initial = list(collection.aggregate(pipeline_q4_initial))
print(json.dumps(result_initial, indent=4, default=json_util.default))
```

```
[
    {
        "_id": {
            "city": "Nappanee",
            "state": "IN"
        },
        "countL": 37,
        "countR": 0,
        "diff": 37,
        "city": "Nappanee"
    },
    {
        "_id": {
            "city": "Florence",
            "state": "OR"
        },
        "countL": 53,
        "countR": 26,
        "diff": 27,
        "city": "Florence"
    }
]
```

Ερώτημα 4

Ζητούμενο: Ο προσδιορισμός της μέγιστης αριθμητικής διαφοράς ατυχημάτων μεταξύ της αριστερής και της δεξιάς πλευράς του δρόμου, εστιάζοντας σε περιπτώσεις

όπου η αριστερή πλευρά υπερτερεί (`Left > Right`), εξαιρώντας όμως τους μονόδρομους.

Τεχνική Υλοποίηση: Η διαδικασία υλοποιήθηκε μέσω ενός Aggregation Pipeline 4 σταδίων:

- Ομαδοποίηση & Pivot (`$group`): Τα δεδομένα ομαδοποιούνται ανά Πόλη και Πολιτεία. Ταυτόχρονα, υπολογίζονται τα αθροίσματα για κάθε πλευρά (`countL` , `countR`) κάνοντας χρήση του τελεστή `$cond` (conditional summation).
- Φιλτράρισμα (`$match`): Εφαρμόζεται σύνθετος έλεγχος μέσω του `$expr` και του λογικού τελεστή `$and` . Επιλέγονται μόνο οι εγγραφές που ικανοποιούν ταυτόχρονα τα εξής:
  - `countL > countR` : Τα ατυχήματα στα αριστερά είναι περισσότερα.
  - `countR > 0` : Τα ατυχήματα στα δεξιά είναι τουλάχιστον 1 (ώστε να αποκλειστούν περιπτώσεις μονόδρομων που στρεβλώνουν το αποτέλεσμα).
- Υπολογισμός Διαφοράς (`$project`): Δημιουργείται το πεδίο `diff` , το οποίο προκύπτει από την αφαίρεση `$countL - $countR` .
- Εξαγωγή Μέγιστης Τιμής (`$group`): Πραγματοποιείται καθολική ομαδοποίηση (`_id: null`) όλων των αποτελεσμάτων για να βρεθεί η απόλυτη μέγιστη τιμή διαφοράς στο dataset, μέσω του τελεστή `$max` .

In [6]:
```
pipeline_q4 = [
    {
        "$group": {
            "_id": {"city": "$city", "state": "$state"},
            "countL": {
                "$sum": {"$cond": [{"$eq": ["$side", "L"]}, 1, 0]}
            },
            "countR": {
                "$sum": {"$cond": [{"$eq": ["$side", "R"]}, 1, 0]}
            }
        }
    },
    {
        "$match": {
            "$expr": {
                "$and": [
                    {"$gt": ["$countL", "$countR"]},
                    {"$gt": ["$countR", 0]}
                ]
            }
        }
    },
    {
        "$project": {
            "diff": {"$subtract": ["$countL", "$countR"]}
        }
    },
    {
        "$group": {
            "_id": None,
```

```
            "maxDiff": {"$max": "$diff"}
        }
    }
]

result_q4 = list(collection.aggregate(pipeline_q4))

print(json.dumps(result_q4, indent=4, default=json_util.default))
```

```
[
    {
        "_id": null,
        "maxDiff": 27
    }
]
```

Ερώτημα 5

Ζητούμενο: Εντοπισμός των 5 ατυχημάτων που καλύπτουν τη μεγαλύτερη απόσταση (`distance(km)`), με ταυτόχρονη προβολή του μέσου όρου απόστασης της πολιτείας στην οποία ανήκουν, καθώς και της απόκλισής τους από αυτόν.

Τεχνική Υλοποίηση: Η υλοποίηση βασίστηκε στον τελεστή `$setWindowFields` , ο οποίος επιτρέπει την εκτέλεση αναλυτικών υπολογισμών σε υποσύνολα εγγράφων, διατηρώντας ταυτόχρονα τις αρχικές εγγραφές (χωρίς να τις ομαδοποιεί όπως το `$group`).

Η διαδικασία περιλαμβάνει τα εξής στάδια:

- Window Function (`$setWindowFields`):
  - Partitioning: Τα δεδομένα διαχωρίζονται λογικά ανά Πολιτεία (`partitionBy: "$state"`).
  - Calculation: Υπολογίζεται ο μέσος όρος (`$avg`) της απόστασης για κάθε partition και η τιμή αυτή προστίθεται ως νέο πεδίο (`averageDistance`) σε κάθε έγγραφο της συγκεκριμένης πολιτείας.
- Ταξινόμηση & Επιλογή (`$sort` , `$limit`): Πραγματοποιείται φθίνουσα ταξινόμηση βάσει απόστασης και διατηρούνται μόνο οι 5 κορυφαίες εγγραφές.
- Μορφοποίηση & Υπολογισμός Διαφοράς (`$project`): Ορίζονται τα τελικά πεδία εμφάνισης και υπολογίζεται δυναμικά η απόκλιση μέσω του τελεστή `$subtract` :

$$Diff = Distance_{accident} - Average_{state}$$

Σημείωση Απόδοσης: Λόγω των απαιτητικών διεργασιών ταξινόμησης σε μεγάλο όγκο δεδομένων, κρίθηκε απαραίτητη η ενεργοποίηση της παραμέτρου `allowDiskUse=True` κατά την εκτέλεση της εντολής `aggregate` , ώστε να επιτραπεί η χρήση προσωρινού χώρου στον δίσκο σε περίπτωση υπέρβασης του ορίου μνήμης RAM.

In [9]:
```
pipeline_q5 = [
    {
        "$setWindowFields": {
```

```
            "partitionBy": "$state",
            "output": {
                "averageDistance": { "$avg": "$distance(km)" }
            }
        }
    },
    { "$sort": {"distance(km)": -1} },
    { "$limit": 5 },
    {
        "$project": {
            "_id": "$state",
            "city": "$city",
            "distance(km)": 1,
            "averageDistance": 1,
            "diffFromAverage": { "$subtract": ["$distance(km)", "$averageDistanc
        }
    }
]
results = list(collection.aggregate(pipeline_q5, allowDiskUse=True))

for doc in results:
    print(json.dumps(doc, indent=4, default=json_util.default))
```

```
{
    "distance(km)": 247.29662707199995,
    "averageDistance": 1.6547963479334853,
    "_id": "MN",
    "city": "Minneapolis",
    "diffFromAverage": 245.64183072406647
}
{
    "distance(km)": 230.52565324800008,
    "averageDistance": 1.5374776903396854,
    "_id": "OH",
    "city": "Van Buren",
    "diffFromAverage": 228.98817555766038
}
{
    "distance(km)": 222.99392332800002,
    "averageDistance": 1.1212391377246196,
    "_id": "OR",
    "city": "Ontario",
    "diffFromAverage": 221.8726841902754
}
{
    "distance(km)": 213.236470656,
    "averageDistance": 1.1692631677325984,
    "_id": "GA",
    "city": "Augusta",
    "diffFromAverage": 212.0672074882674
}
{
    "distance(km)": 161.05671014400002,
    "averageDistance": 1.1212391377246196,
    "_id": "OR",
    "city": "Ontario",
    "diffFromAverage": 159.9354710062754
}
```